



Sri Lanka Institute of Information Technology

HarvestIQ System

Project Report

Fundamentals of Data Mining - 2025

Group Name: Mining Minds

Submitted by:

1. IT23247390 – T.A.O.S Jayathilaka
2. IT23214248 – H.N Ranaweera
3. IT23218680 – Perera N.T
4. IT23190498 – S. Vashika

Submitted to:

Dr Prasanna Sumathipala

Name of the supervisor/lecturer in charge

05/10/2025- Date of submission

Video link : <https://drive.google.com/file/d/1-aK2mmymwIyMtUzYnEJ091k2zVkIdjng/view?usp=sharing>

Deployed link:

https://huggingface.co/spaces/hnr-2002/sl_yield_predictor

GitHub link :

https://github.com/HimayaRanaweera/sl_yield_predictor.git

Contents

1.Declaration.....	3
2.Acknowledgment	3
3.Background	3
4.Introduction and Problem Definition.....	4
5.Objectives	5
6.Methodology.....	6
6.1 Data Gathering.....	7
6.2 Data Preprocessing	8
6.3 Data Analyzing	13
6.4 Data Visualizations.....	14
6.4.1 Yield by Crop	14
6.4.2 Yield vs Rainfall.....	15
6.4.3Average Yield by Rainfall bins.....	16
6.4.4 Yield vs Fertilizer	17
6.4.5 Yield vs Temperature	18
6.4.6 Boxplot by Region/Province	19
6.5 Correlation Heatmap.....	20
6.6 Train-test Split.....	21
6.7 Hyperparameter Tuning	25
6.8 Predictions	27
7. User Interfaces.....	31
8.Conclusion	32

1.Declaration

We declare that this report is the result of our own work. All external sources used for methodology inspiration and section structure are properly acknowledged and cited. The analyses, figures, and interpretations presented here are original and based on the “FDMSriLanka_Agri_Yield_Raw_20k.xlsx” dataset.

2.Acknowledgment

We are grateful to our course instructors and peers for their guidance on data-mining best practices, and to the organizations that compile and maintain Sri Lankan agriculture statistics. Their support and resources enabled the successful completion of this project.

3.Background

Agricultural yield prediction is vital for food security, pricing, input planning, and supply-chain decisions. In Sri Lanka’s two-season context (Maha and Yala), yields depend on weather (rainfall, temperature), agronomy (area sown/harvested, irrigation, fertilizer), location (province/district), soil characteristics, and crop type. Accurate, early-season predictions help ministries plan imports/exports, guide fertilizer allocations, and support farmers with timely advisories.

Our dataset contains 20,100 records (2010–2024) across 15 variables describing climate, management, geography, and outcomes (yield/production and market price). It lets us model yield before the season ends and assess how controllable (fertilizer, area sown) and uncontrollable (rainfall) factors drive outcomes at district level.

4.Introduction and Problem Definition

In a climate-sensitive and resource-constrained agricultural landscape, Sri Lanka's food security and farmer livelihoods hinge on timely, accurate yield expectations. Decisions on seed distribution, fertilizer allocation, irrigation scheduling, and market operations must be made weeks—often months—before harvest. Yet yield outcomes are shaped by many moving parts: seasonality (Maha/Yala), regional weather patterns, soil characteristics, irrigation method, cultivated area, and crop-specific responses to inputs like fertilizer. Data gaps, late reporting, and local variability across provinces and districts further complicated planning.

As a result, ministries, provincial officers, and farmer organizations frequently operate with uncertainty. Overestimation risks surplus handling costs and depressed farm-gate prices; underestimation triggers avoidable imports and supply shocks. A structured, data-driven approach to early-season yield prediction can convert raw field, climate, and management data into actionable foresight.

Problem Definition

Core problem : Can we reliably predict expected yield (mt/ha) or total production (mt) for key crops before the season ends using inputs available during the growing season (e.g., cumulative rainfall to date, average temperature, area sown, soil type, irrigation, and fertilizer application)?

Key challenges include:

1. For planners (ministries/provincial offices):
 - Difficulty aligning input provisioning (fertilizer, irrigation water, machinery) and trade decisions with actual end-of-season outcomes.
 - Risk of costly misallocation when forecasts are late or imprecise.
2. For farmers and extension services:
 - Limited visibility into expected yield under current weather and management, making it harder to adjust practices mid-season (e.g., top-dressing fertilizer, supplemental irrigation).

3. For markets and supply chains:

- Price volatility driven by uncertain production volumes; logistical inefficiencies in storage, transport, and processing due to inaccurate yield signals.

4. For data quality and integration:

- Mixed data types (e.g., numeric values stored as text), missing observations (rainfall, fertilizer), and heterogeneity across crops, soils, and districts that can bias naïve models.

Proposed solution (project scope).

We use the multi-season Sri Lankan agriculture dataset (crop, season, province/district, soil type, irrigation, area sown/harvested, rainfall, temperature, fertilizer, market price) to train regression models that predict the continuous target Yield_mt_per_ha (and optionally Production_mt). This delivers early, quantitative yield forecasts to support input planning and market decisions.

5.Objectives

Primary goal: Build a robust regression model that predicts crop yield (mt/ha) accurately and early in the season, using Sri Lankan multi-season data (crop, season, province/district, soil type, irrigation, area sown/harvested, rainfall, temperature, fertilizer, market price). The model should deliver actionable insights for planning, risk management, and resource allocation.

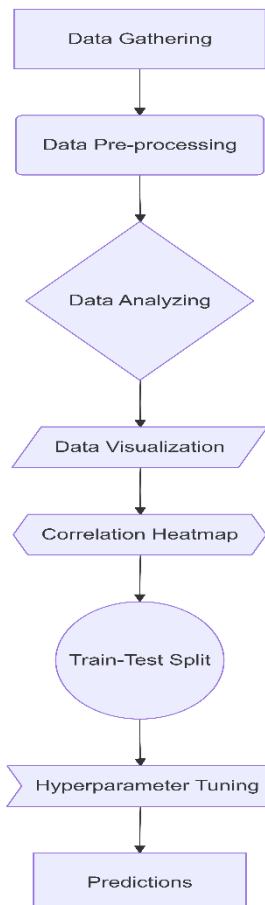
This can help:

- Government & Planners:
Optimize fertilizer and water allocations, plan imports/exports, and set realistic production targets at province/district level.
- Farmers & Extension Services:
Gain early yield expectations to adjust practices (e.g., top-dressing, supplemental irrigation) and reduce losses.
- Agribusiness & Markets:
Improve procurement, storage, and logistics planning; reduce price volatility through better supply visibility.

- Researchers & Analysts:
Understand key yield drivers across crops and regions; quantify the impact of weather, irrigation, soils, and inputs.
- Food Security & Policy Makers:
Support evidence-based interventions and contingency planning during adverse seasons (drought/flood).

6.Methodology

- 1.Data Gathering
2. Data Pre-processing
3. Data analyzing
4. Data visualization
5. Correlation Heatmap
6. Train-test split
7. Hyper parameter tuning
- 8.Predictions



6.1 Data Gathering

The dataset we selected is the Sri Lanka Agriculture Yield dataset.
The raw data contains 20,100 rows (records) and 15 columns (features).

Link to the dataset: [SriLanka_Agri_Yield_Raw_20k.xlsx](#)

Variable List :

Variable Name	Description	Variable Type
Year	Calendar year of the record	Integer
Season	Cultivation season (Maha or Yala)	Categorical
Province	Province within Sri Lanka	Categorical
District	District within the province	Categorical
Crop	Crop type (e.g., Paddy, Maize, Onion)	Categorical
Soil_Type	Dominant soil category for the area	Categorical
Irrigation	Water source/method (Irrigated or Rainfed)	Categorical
Area_Sown_ha	Area sown at the start of season (hectares)	Numeric (stored as text)
Area_Harvested_ha	Area actually harvested (hectares)	Numeric
Rainfall_mm	Seasonal/cumulative rainfall (millimeters)	Numeric (stored as text)
Temperature_C	Average/seasonal temperature (°C)	Numeric
Fertilizer_kg_per_ha	Fertilizer application rate (kg/ha)	Numeric (stored as text)
Yield_mt_per_ha	Yield per hectare (metric tons/ha)	Numeric
Production_mt	Total production (metric tons)	Numeric
Market_Price_LKR_per_kg	Market price (LKR per kg)	Numeric (stored as text)

6.2 Data Preprocessing

Import Libraries

```
▶ # full_pipeline_with_eda_models_and_tuning.py
# -----
# Requirements:
# - pandas, numpy, matplotlib, seaborn
# - scikit-learn
# - xgboost, lightgbm, catboost (optional; script checks and uses if installed)
# -----

import warnings
warnings.filterwarnings("ignore")
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import joblib
import time

▶ from sklearn.model_selection import train_test_split, RandomizedSearchCV, cross_val_score
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.compose import TransformedTargetRegressor
from sklearn.metrics import r2_score, mean_squared_error
from sklearn.inspection import permutation_importance

# Models
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor, ExtraTreesRegressor
from sklearn.svm import SVR

# Optional advanced boosters
try:
    from xgboost import XGBRegressor
    xgb_available = True
except Exception:
    xgb_available = False

try:
    from lightgbm import LGBMRegressor
    lgbm_available = True
except Exception:
    lgbm_available = False

try:
    from catboost import CatBoostRegressor
    catboost_available = True
except Exception:
    catboost_available = False
```


This part imports all essential Python libraries for data processing, visualization, and machine learning. It also conditionally loads advanced boosting algorithms (XGBoost, LightGBM, CatBoost) if installed.

Load Dataset

```
# 1) Load & Basic Cleaning

FILE = "/content/SriLanka_Agri_Yield_Raw_20k.xlsx" # change if needed
df = pd.read_excel(FILE)
df = df.drop_duplicates()
```

This section loads the agricultural yield dataset from an Excel file and removes duplicate rows to ensure clean and consistent data for analysis.

Remove Duplicates

```
# Drop columns we typically don't want for yield prediction
drop_cols = ['id', 'transaction_id', 'Market_Price_LKR_per_kg']
df = df.drop(columns=[c for c in drop_cols if c in df.columns], errors='ignore')
```

Unnecessary or unrelated columns (such as IDs or market prices) are dropped to focus only on features relevant to predicting crop yield.

```
# unify missing-value tokens
df.replace(['unknown', 'missing', 'NA', 'na'], np.nan, inplace=True)
```

This replaces different forms of missing value indicators with a standard NaN format, making it easier to handle missing data later in preprocessing.

```

▶ # coerce numeric candidates
num_candidates = ['Fertilizer_kg_per_ha', 'Rainfall_mm', 'Area_Sown_ha',
                  'Area_Harvested_ha', 'Production_mt', 'Temperature_C', 'Yield_mt_per_ha']
for c in num_candidates:
    if c in df.columns:
        df[c] = pd.to_numeric(df[c], errors='coerce')

```

Ensures all key numeric columns are properly converted to numeric data types, automatically converting invalid entries to NaN for later imputation.

```

▶ # quick glance
print("Rows / Cols:", df.shape)
print("Columns:", df.columns.tolist())

```

```

↗ Rows / Cols: (20000, 14)
Columns: ['Year', 'Season', 'Province', 'District', 'Crop', 'Soil_Type', 'Irrigation', 'Area_Sc

```

Displays the dataset's basic structure — total number of rows, columns, and column names — to confirm successful loading and initial cleaning.

```

▶ # 2) Fill missing and feature-engineer

# Target name
TARGET = 'Yield_mt_per_ha'
if TARGET not in df.columns:
    raise ValueError(f"Target column '{TARGET}' not found in the dataset.")

```

This section defines the target variable (Yield_mt_per_ha) for prediction and checks whether it exists in the dataset — if not, it raises an error to prevent the code from running with a missing or incorrect target column.

Encode categorical variables

```
# fill numeric with median and categorical with mode
num_cols = df.select_dtypes(include=np.number).columns.tolist()
cat_cols = df.select_dtypes(exclude=np.number).columns.tolist()
if TARGET in num_cols:
    num_cols.remove(TARGET)

for c in num_cols:
    df[c] = df[c].fillna(df[c].median())

for c in cat_cols:
    # if empty mode can throw; guard against no values
    if df[c].isna().all():
        df[c] = df[c].fillna("Unknown")
    else:
        df[c] = df[c].fillna(df[c].mode()[0])
```

This section fills missing numeric values with the median and categorical ones with the mode to prevent data loss and maintain statistical consistency.

```
# Fix inconsistent Production using Yield * Area_Harvested_ha
if {'Production_mt', 'Area_Harvested_ha', TARGET}.issubset(df.columns):
    calc = df[TARGET] * df['Area_Harvested_ha']
    mask = df['Production_mt'].isna() | (abs(df['Production_mt'] - calc) > 0.1 * (calc + 1e-6))
    df.loc[mask, 'Production_mt'] = calc[mask]
```

Checks and recalculates inconsistent production values using the formula Yield Area_Harvested, ensuring logical consistency between related variables.

```
▶ # Outlier removal on target using IQR
q1, q3 = df[TARGET].quantile([0.25, 0.75])
iqr = q3 - q1
lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr
orig_len = len(df)
df = df[(df[TARGET] >= lower) & (df[TARGET] <= upper)]
print(f"Removed {orig_len - len(df)} outlier rows based on {TARGET} (IQR).")
```

```
🔄 Removed 3218 outlier rows based on Yield_mt_per_ha (IQR).
```

Removes extreme outlier rows in the target variable (Yield) using the Interquartile Range (IQR) method to improve model reliability and reduce skewness.

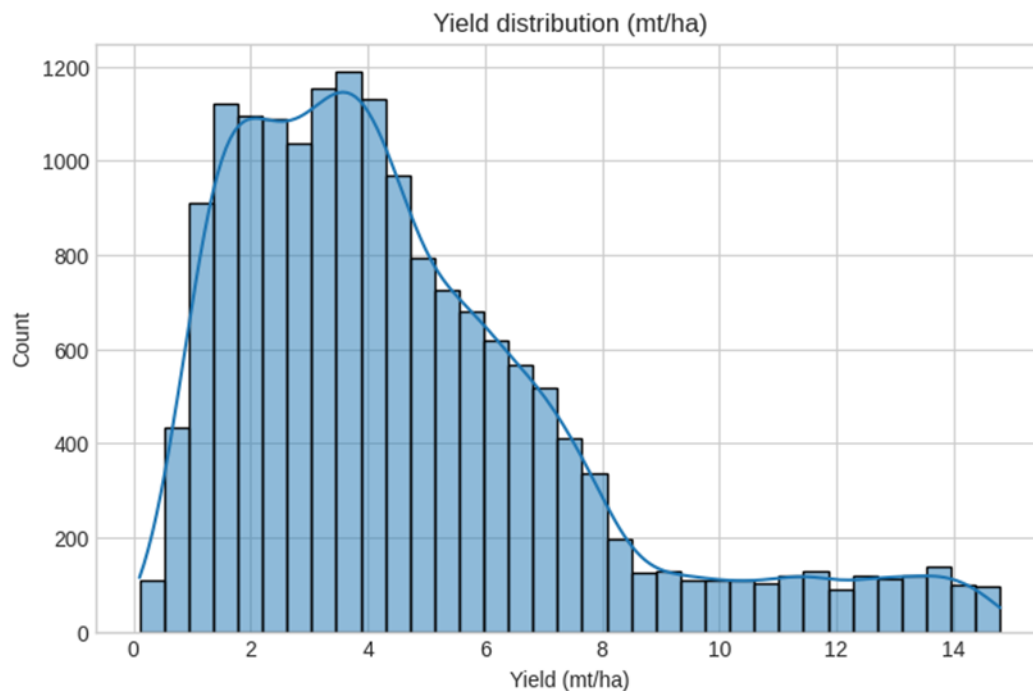
6.3 Data Analyzing



```
# 3) Exploratory Visualizations (EDA)

plt.style.use('seaborn-v0_8-whitegrid')

# Distribution of Yield
plt.figure(figsize=(8,5))
sns.histplot(df[TARGET], kde=True, bins=35)
plt.title("Yield distribution (mt/ha)")
plt.xlabel("Yield (mt/ha)")
plt.ylabel("Count")
plt.show()
```

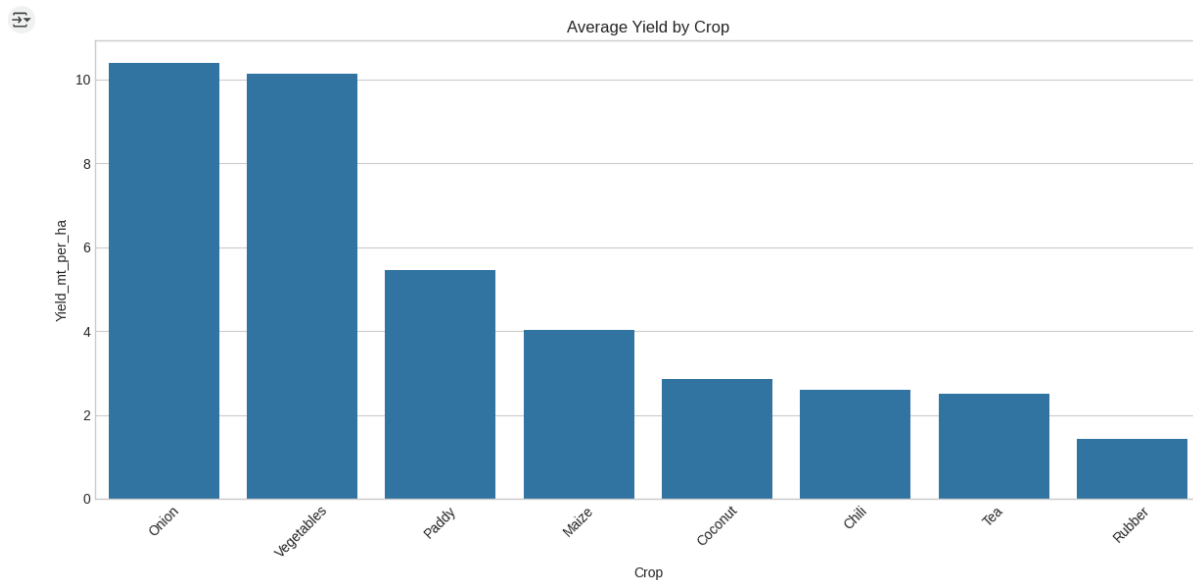


Plots the distribution of the crop yield variable to understand its spread, central tendency, and possible skewness in the data

6.4 Data Visualizations

6.4.1 Yield by Crop

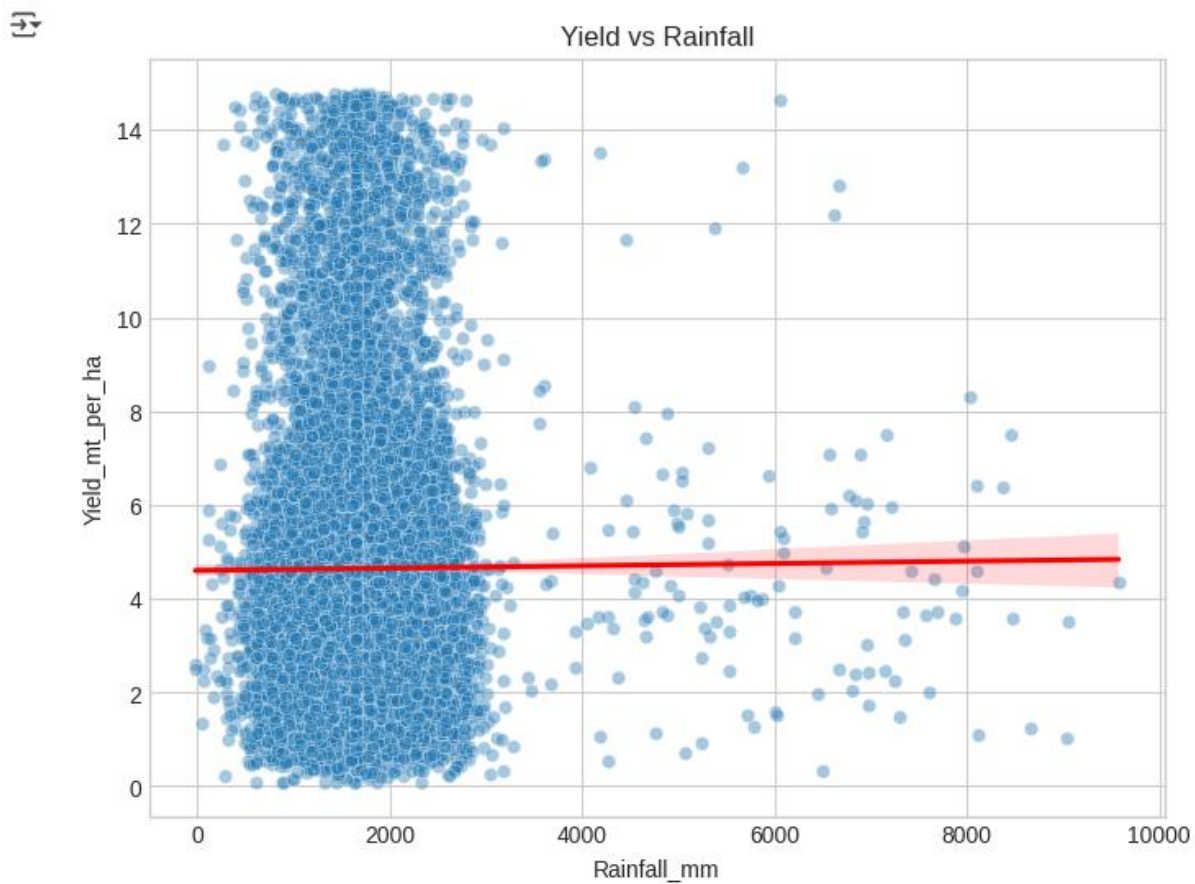
```
# Yield by Crop (if available)
if 'Crop' in df.columns:
    plt.figure(figsize=(12,6))
    order = df.groupby('Crop')[TARGET].mean().sort_values(ascending=False).index
    sns.barplot(x='Crop', y=TARGET, data=df, order=order, ci=None)
    plt.xticks(rotation=45)
    plt.title("Average Yield by Crop")
    plt.tight_layout()
    plt.show()
```



Shows a bar chart comparing average yield across different crop types to identify which crops have higher or lower productivity.

6.4.2 Yield vs Rainfall

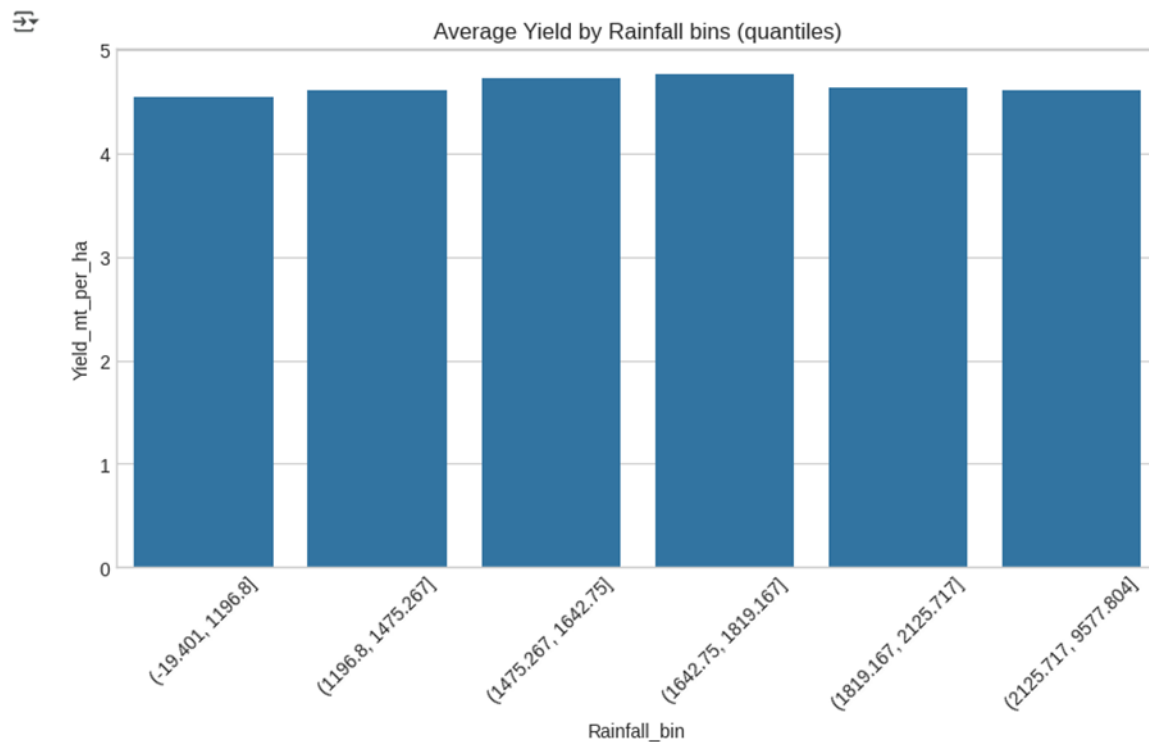
```
# Yield vs Rainfall scatter + regression
if 'Rainfall_mm' in df.columns:
    plt.figure(figsize=(8,6))
    sns.scatterplot(data=df, x='Rainfall_mm', y=TARGET, alpha=0.4)
    sns.regplot(data=df, x='Rainfall_mm', y=TARGET, scatter=False, color='red')
    plt.title("Yield vs Rainfall")
    plt.show()
```



Displays the relationship between rainfall and yield through scatter and regression plots, helping assess how precipitation levels affect crop performance.

6.4.3 Average Yield by Rainfall bins

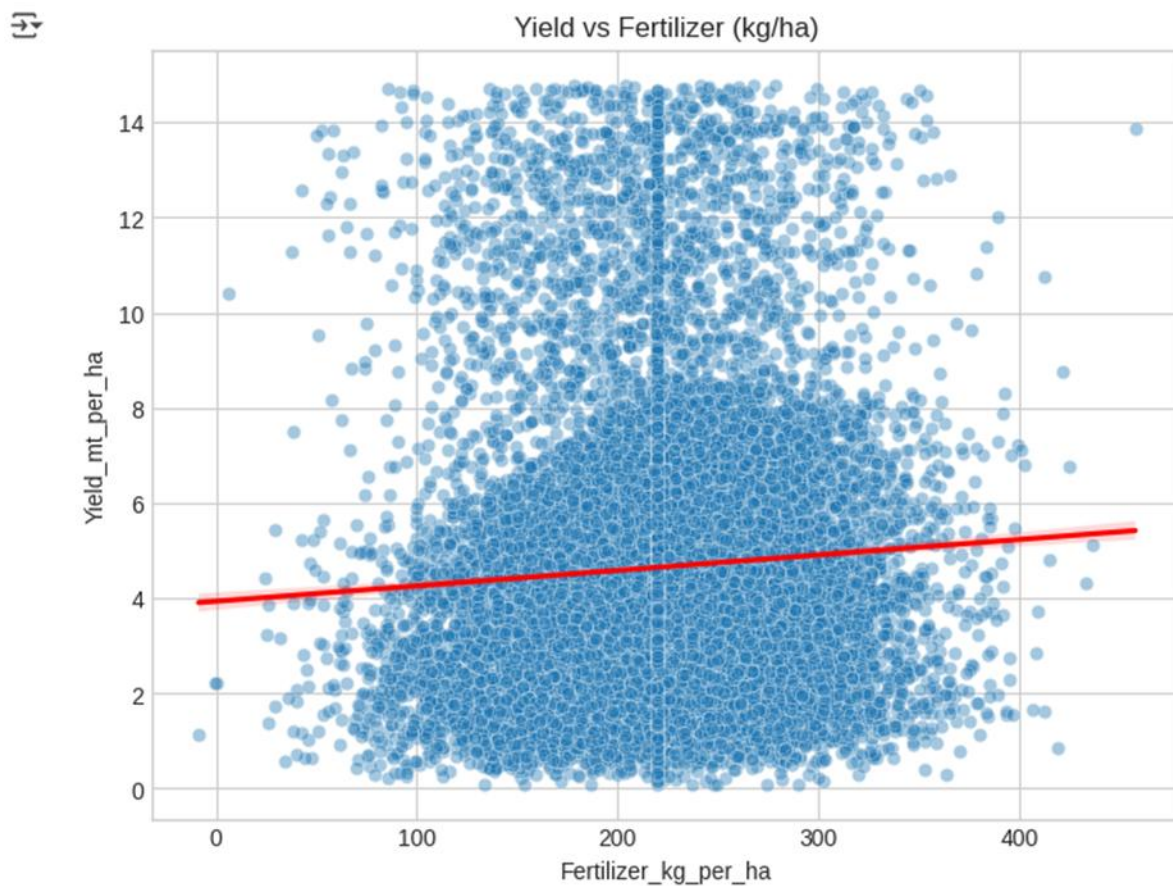
```
# binned rainfall -> mean yield
if 'Rainfall_mm' in df.columns:
    df['Rainfall_bin'] = pd.qcut(df['Rainfall_mm'], q=6, duplicates='drop')
    plt.figure(figsize=(10,5))
    sns.barplot(x='Rainfall_bin', y=TARGET, data=df, estimator=np.mean, ci=None)
    plt.xticks(rotation=45)
    plt.title("Average Yield by Rainfall bins (quantiles)")
    plt.show()
    df.drop(columns=['Rainfall_bin'], inplace=True)
```



Groups rainfall into quantile bins to visualize how different rainfall levels influence average yield, revealing patterns in water dependency.

6.4.4 Yield vs Fertilizer

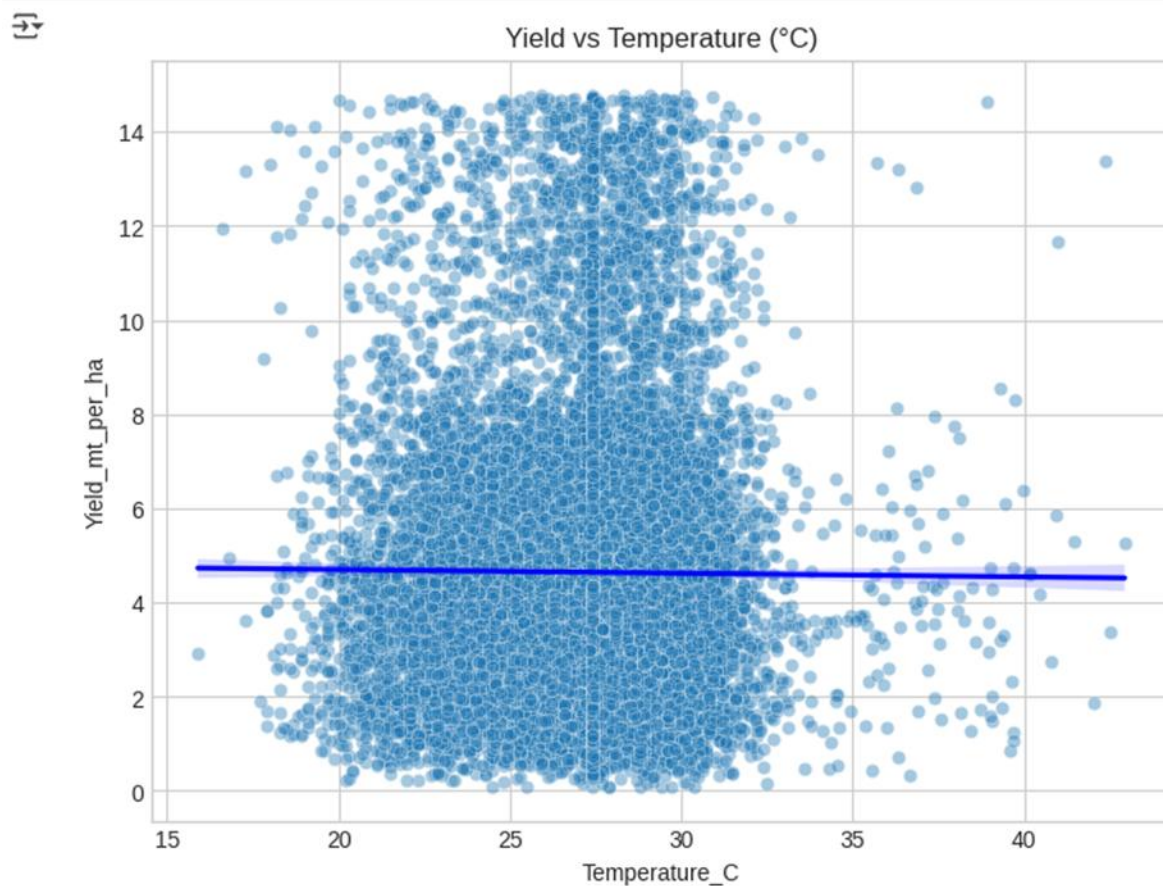
```
▶ # Yield vs Fertilizer
if 'Fertilizer_kg_per_ha' in df.columns:
    plt.figure(figsize=(8,6))
    sns.scatterplot(data=df, x='Fertilizer_kg_per_ha', y=TARGET, alpha=0.4)
    sns.regplot(data=df, x='Fertilizer_kg_per_ha', y=TARGET, scatter=False, color='red')
    plt.title("Yield vs Fertilizer (kg/ha)")
    plt.show()
```



Visualizes the relationship between fertilizer usage and crop yield, helping identify the point where fertilizer input optimizes productivity.

6.4.5 Yield vs Temperature

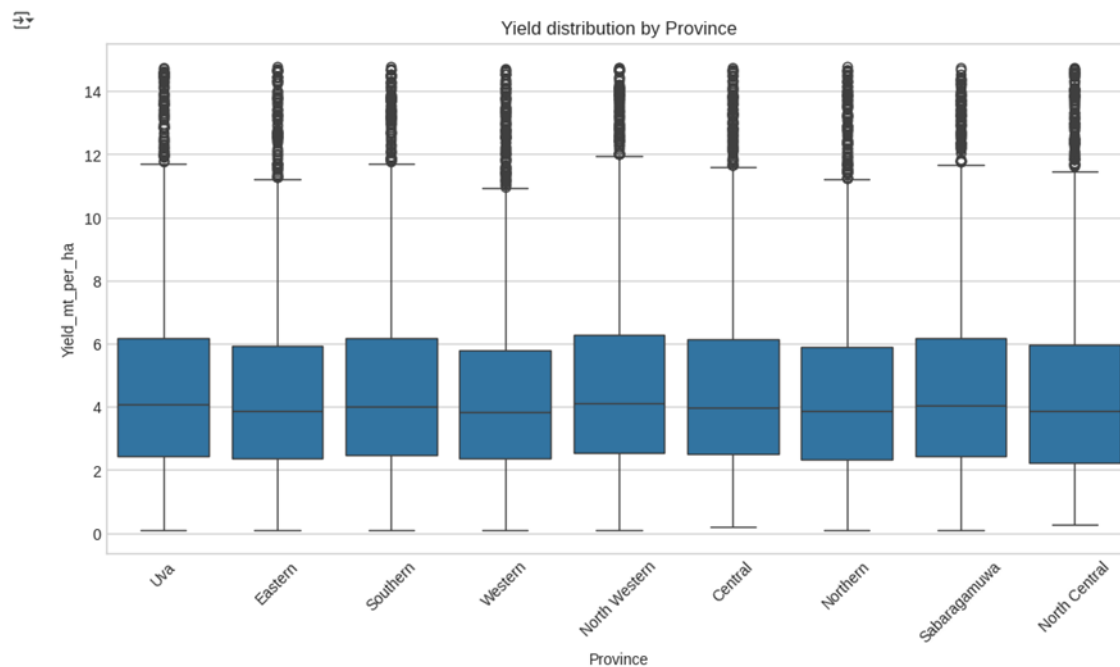
```
▶ # Yield vs Temperature
if 'Temperature_C' in df.columns:
    plt.figure(figsize=(8,6))
    sns.scatterplot(data=df, x='Temperature_C', y=TARGET, alpha=0.4)
    sns.regplot(data=df, x='Temperature_C', y=TARGET, scatter=False, color='blue')
    plt.title("Yield vs Temperature (°C)")
    plt.show()
```



Examines how temperature variations influence crop yield, showing whether extreme or moderate temperatures are more favorable for production.

6.4.6 Boxplot by Region/Province

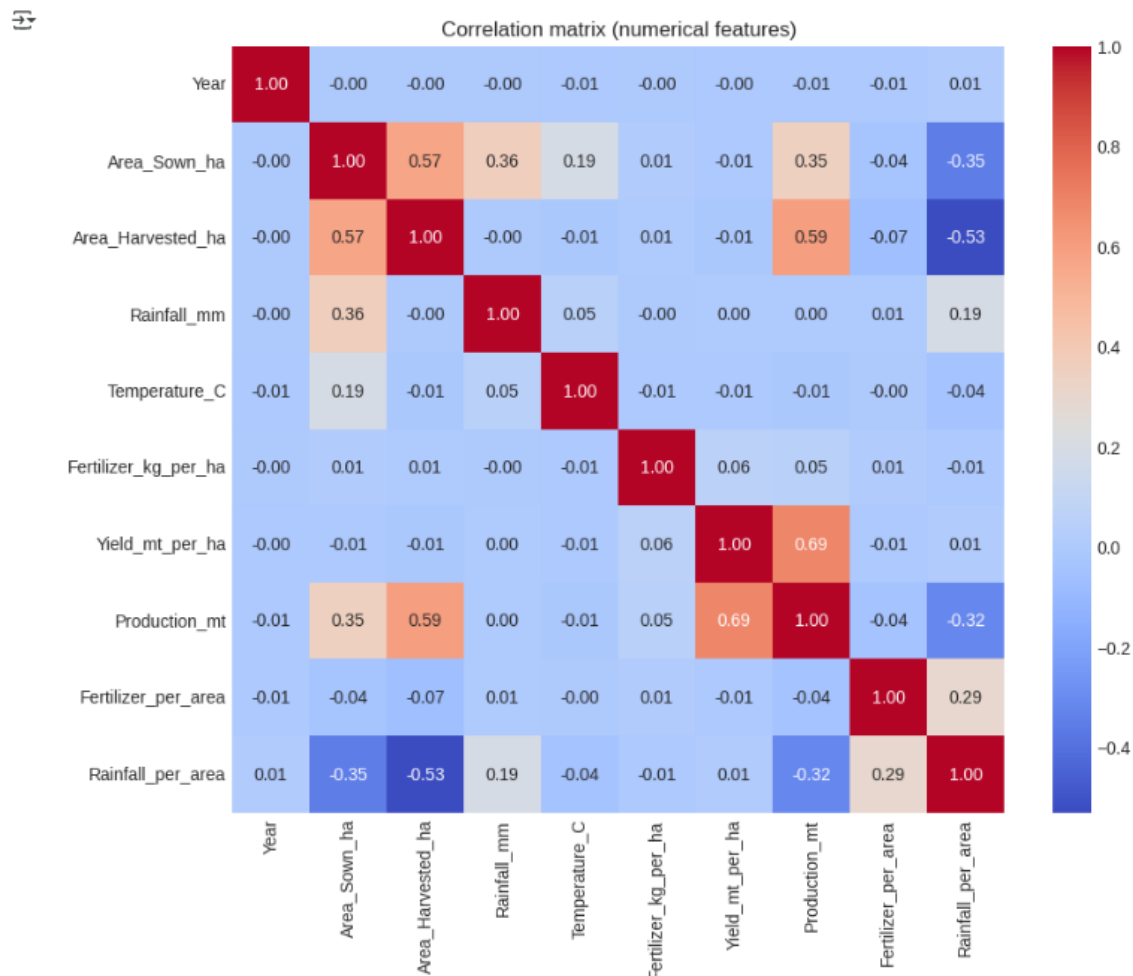
```
# Boxplot by Region/Province if present
for region_col in ['Region', 'Province', 'District']:
    if region_col in df.columns:
        plt.figure(figsize=(12,6))
        sns.boxplot(x=region_col, y=TARGET, data=df)
        plt.xticks(rotation=45)
        plt.title(f"Yield distribution by {region_col}")
        plt.show()
    break
```



Compares yield distributions across regions or provinces to identify geographic areas with higher productivity or wider variation in yield.

6.5 Correlation Heatmap

```
# Correlation heatmap (numerical features)
num_cols_present = df.select_dtypes(include=np.number).columns.tolist()
if TARGET not in num_cols_present:
    num_cols_present.append(TARGET)
plt.figure(figsize=(10,8))
sns.heatmap(df[num_cols_present].corr(), annot=True, fmt=".2f", cmap='coolwarm')
plt.title("Correlation matrix (numerical features)")
plt.show()
```



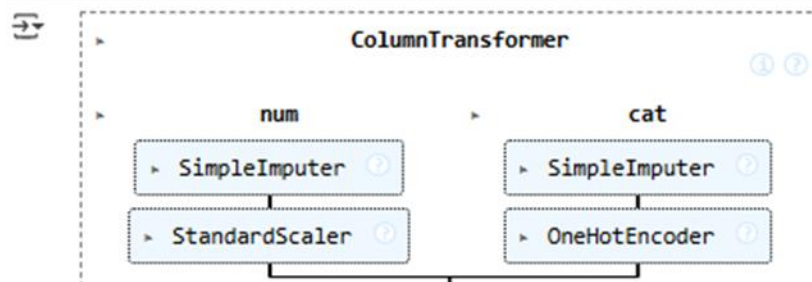
Generates a correlation heatmap to visualize relationships between numerical variables, helping identify which features strongly influence yield.

6.6 Train-test Split

```
# =====  
# 4) Preprocessing + train/test split  
# =====  
X = df.drop(columns=[TARGET])  
y = df[TARGET].copy()  
  
num_cols = X.select_dtypes(include=np.number).columns.tolist()  
cat_cols = X.select_dtypes(exclude=np.number).columns.tolist()  
  
numeric_transformer = Pipeline(steps=[  
    ('imputer', SimpleImputer(strategy='median')),  
    ('scaler', StandardScaler())  
)  
  
categorical_transformer = Pipeline(steps=[  
    ('imputer', SimpleImputer(strategy='most_frequent')),  
    ('onehot', OneHotEncoder(handle_unknown='ignore'))  
)  
  
preprocessor = ColumnTransformer(transformers=[  
    ('num', numeric_transformer, num_cols),  
    ('cat', categorical_transformer, cat_cols)  
, remainder='drop']
```

Sets up preprocessing pipelines to scale numeric features and encode categorical ones, ensuring all data is properly formatted for model training.

```
# Fit preprocessor separately so we can get feature names later  
preprocessor.fit(X) # cheap compared to model training
```



Fits the preprocessing steps to the dataset to capture transformation parameters (like median and category mappings) before applying models.

```
▶ # get transformed feature names
num_feat_names = num_cols
cat_encoder = preprocessor.named_transformers_['cat'].named_steps['onehot']
try:
    cat_feat_names = list(cat_encoder.get_feature_names_out(cat_cols))
except:
    # fallback
    cat_feat_names = [f"{c}__{i}" for c in cat_cols for i in range(3)]
feature_names = num_feat_names + cat_feat_names
print("Total model features after preprocessing:", len(feature_names))
```

```
➞ Total model features after preprocessing: 60
```

Collects the final list of feature names after preprocessing, including one-hot encoded categories, to interpret feature importance later.

```
# train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)
print("Train / Test:", X_train.shape, X_test.shape)
```

```
➞ Train / Test: (13425, 15) (3357, 15)
```

Splits the dataset into training and testing subsets, allowing model performance to be evaluated on unseen data.

```

# =====
# 5) Baseline models (quick) – train & store predictions
# =====
def build_models_dict():
    models = {}
    # Keep transformed target for linear and SVR to enforce positivity
    models["Linear Regression"] = TransformedTargetRegressor(
        regressor=Pipeline([('preproc', preprocessor), ('lr', LinearRegression())]),
        func=np.log1p, inverse_func=np.expm1
    )
    models["Decision Tree"] = Pipeline([('preproc', preprocessor), ('reg', DecisionTreeRegressor(random_state=42))])
    models["Random Forest"] = Pipeline([('preproc', preprocessor), ('reg', RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1))])
    models["Extra Trees"] = Pipeline([('preproc', preprocessor), ('reg', ExtraTreesRegressor(n_estimators=100, random_state=42, n_jobs=-1))])
    models["Gradient Boosting"] = Pipeline([('preproc', preprocessor), ('reg', GradientBoostingRegressor(random_state=42))])
    # SVR wrapped to predict positive via log transform
    models["SVR"] = TransformedTargetRegressor(
        regressor=Pipeline([('preproc', preprocessor), ('svr', SVR(kernel='rbf'))]),
        func=np.log1p, inverse_func=np.expm1
    )

    if xgb_available:
        models["XGBoost"] = Pipeline([('preproc', preprocessor), ('reg', XGBRegressor(n_estimators=200, random_state=42, verbosity=0, n_jobs=-1))])
    if lgbm_available:
        models["LightGBM"] = Pipeline([('preproc', preprocessor), ('reg', LGBMRegressor(n_estimators=200, random_state=42, n_jobs=-1))])
    if catboost_available:
        models["CatBoost"] = Pipeline([('preproc', preprocessor), ('reg', CatBoostRegressor(iterations=200, random_state=42, verbose=0))])

    return models

```

This section defines multiple baseline regression models (like Linear, Random Forest, XGBoost, etc.) in a function so they can be easily trained and compared later, helping to identify which algorithm performs best for yield prediction.

```

models = build_models_dict()

baseline_results = {}
predictions_by_model = {}

for name, model in models.items():
    t0 = time.time()
    print(f"\nTraining baseline: {name} ...")
    model.fit(X_train, y_train)
    train_time = time.time() - t0

    # Predict (clip negatives to 0)
    y_pred = model.predict(X_test)
    y_pred = np.clip(y_pred, 0, None)

    r2 = r2_score(y_test, y_pred)
    mse = mean_squared_error(y_test, y_pred)
    baseline_results[name] = {'R2': r2, 'MSE': mse, 'train_time_s': train_time}
    predictions_by_model[name] = y_pred

    print(f"{name}: R2={r2:.4f}, MSE={mse:.4f}, train_time={train_time:.1f}s")

# Create summary table
results_df = pd.DataFrame(baseline_results).T.sort_values(by='R2', ascending=False)
print("\nBaseline model ranking (by R²):")
print(results_df)

```



```
Training baseline: Linear Regression ...
Linear Regression: R2=0.7613, MSE=2.1805, train_time=0.3s

Training baseline: Decision Tree ...
Decision Tree: R2=0.5462, MSE=4.1460, train_time=3.7s

Training baseline: Random Forest ...
Random Forest: R2=0.7631, MSE=2.1642, train_time=99.8s

Training baseline: Extra Trees ...
Extra Trees: R2=0.7450, MSE=2.3294, train_time=121.9s

Training baseline: Gradient Boosting ...
Gradient Boosting: R2=0.7733, MSE=2.0716, train_time=4.5s

Training baseline: SVR ...
SVR: R2=0.7217, MSE=2.5428, train_time=27.0s

Training baseline: XGBoost ...
XGBoost: R2=0.7327, MSE=2.4423, train_time=0.8s

Training baseline: LightGBM ...
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.000668 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 1393
[LightGBM] [Info] Number of data points in the train set: 13425, number of used features: 57
[LightGBM] [Info] Start training from score 4.652540
LightGBM: R2=0.7636, MSE=2.1599, train_time=0.5s

Baseline model ranking (by R²):
      R2      MSE  train_time_s
Gradient Boosting  0.773254  2.071567    4.503188
LightGBM          0.763580  2.159948    0.480100
Random Forest     0.763116  2.164192   99.834478
Linear Regression  0.761333  2.180482    0.302334
Extra Trees       0.745037  2.329365  121.858618
XGBoost           0.732674  2.442313    0.818576
SVR               0.721671  2.542833   26.975153
Decision Tree     0.546196  4.145988    3.690142
```

This section trains all baseline models, evaluates them using R^2 and MSE, and records their training time and predictions, then creates a summary table to compare model performance and identify the best one.

6.7 Hyperparameter Tuning

```
# =====
# 6) Choose best baseline model and hyper-tune it
# =====
best_baseline_name = results_df.index[0]
print(f"\nBest baseline model: {best_baseline_name}")

# We'll tune if the selected model is one of the tunable set (trees/boosters/SVR)
tunable_tree_models = {"Random Forest", "Extra Trees", "Gradient Boosting", "Decision Tree", "XGBoost", "LightGBM", "CatBoost"}
tunable_svr = {"SVR"}

# helper to build a fresh pipeline with a given sklearn-compatible estimator instance
def pipeline_with_estimator(estimator):
    return Pipeline([('preproc', preprocessor), ('reg', estimator)])

# define parameter grids for RandomizedSearchCV (reasonable defaults)
param_grids = {
    "Random Forest": {
        'reg_n_estimators': [100, 200, 400, 700],
        'reg_max_depth': [6, 10, 20, None],
        'reg_min_samples_split': [2, 5, 10],
        'reg_max_features': ['auto', 'sqrt', 0.5]
    },
    "Extra Trees": {
        'reg_n_estimators': [100, 200, 400],
        'reg_max_depth': [6, 10, 20, None],
        'reg_min_samples_split': [2, 5, 10],
    },
    "Gradient Boosting": {
        'reg_n_estimators': [100, 200, 400],
        'reg_learning_rate': [0.01, 0.05, 0.1],
        'reg_max_depth': [3, 6, 10],
    },
    "Decision Tree": {
        'reg_max_depth': [5, 10, 20, None],
        'reg_min_samples_split': [2, 5, 10],
    }
}

if xgb_available:
    param_grids["XGBoost"] = {
        'reg_n_estimators': [100, 300, 500],
        'reg_max_depth': [3, 6, 10],
        'reg_learning_rate': [0.01, 0.05, 0.1],
        'reg_subsample': [0.6, 0.8, 1.0],
        'reg_colsample_bytree': [0.6, 0.8, 1.0]
    }

if lgbm_available:
    param_grids["LightGBM"] = {
        'reg_n_estimators': [100, 300, 500],
        'reg_max_depth': [6, 10, 20, -1],
        'reg_learning_rate': [0.01, 0.05, 0.1],
        'reg_num_leaves': [31, 60, 100]
    }

if catboost_available:
    param_grids["CatBoost"] = {
        'reg_iterations': [100, 300, 500],
        'reg_depth': [4, 6, 10],
        'reg_learning_rate': [0.01, 0.05, 0.1]
    }

# SVR tuning grid (we'll tune on log(target) to be safe)
svr_param_grid = {
    'reg_svr__C': [0.1, 1, 10],
    'reg_svr__epsilon': [0.01, 0.05, 0.1],
    'reg_svr__gamma': ['scale', 'auto']
}

# prepare tune target & pipeline based on best model
tuned = False
best_model_tuned = None
if best_baseline_name in tunable_tree_models:
    print(f"Tuning {best_baseline_name} with RandomizedSearchCV ...")
    if best_baseline_name == "Random Forest":
        estimator = RandomForestRegressor(random_state=42, n_jobs=-1)
    elif best_baseline_name == "Extra Trees":
        estimator = ExtraTreesRegressor(random_state=42, n_jobs=-1)
    elif best_baseline_name == "Gradient Boosting":
        estimator = GradientBoostingRegressor(random_state=42)
    elif best_baseline_name == "Decision Tree":
        estimator = DecisionTreeRegressor(random_state=42)
    elif best_baseline_name == "XGBoost" and xgb_available:
        estimator = XGBRegressor(random_state=42, verbosity=0, n_jobs=-1)
    elif best_baseline_name == "LightGBM" and lgbm_available:
        estimator = LGBMRegressor(random_state=42, n_jobs=-1)
    elif best_baseline_name == "CatBoost" and catboost_available:
        estimator = CatBoostRegressor(random_state=42, verbose=0)
    else:
        estimator = RandomForestRegressor(random_state=42, n_jobs=-1) # fallback

    tune_pipeline = Pipeline([('preproc', preprocessor), ('reg', estimator)])
    param_dist = param_grids.get(best_baseline_name, param_grids.get("Random Forest"))

    rs = RandomizedSearchCV(
        tune_pipeline, param_distributions=param_dist,
        n_iter=20, cv=3, scoring='r2', random_state=42, n_jobs=-1, verbose=1
    )
```

```

rs.fit(X_train, y_train)
best_model_tuned = rs.best_estimator_
print("Tuning complete. Best params:", rs.best_params_)
tuned = True

elif best_baseline_name in tunable_svr:
    print("Tuning SVR on log-transformed target ...")
    # We'll tune SVR by applying log-transformation to y (so we search on log space)
    y_train_log = np.log1p(y_train)
    svr_pipeline = Pipeline([('preproc', preprocessor), ('svr', SVR())])
    rs = RandomizedSearchCV(
        svr_pipeline, param_distributions={
            'svr__C': [0.1, 1, 10, 50],
            'svr__epsilon': [0.01, 0.05, 0.1, 0.2],
            'svr__gamma': ['scale', 'auto']
        }, n_iter=15, cv=3, scoring='r2', random_state=42, n_jobs=-1, verbose=1
    )
    rs.fit(X_train, y_train_log)
    # We'll wrap best estimator with a log-target inversion for predictions
    best_svr_pipeline = rs.best_estimator_
    best_model_tuned = TransformedTargetRegressor(regressor=best_svr_pipeline, func=np.log1p, inverse_func=np.expml)
    best_model_tuned.regressor_ = best_svr_pipeline # ensure attribute for sklearn
    tuned = True
    print("SVR tuning complete. Best params:", rs.best_params_)

else:
    # fallback: tune RandomForest even if it wasn't the top baseline (user insisted tuning)
    print("Best baseline is not in the primary tunable set. As a fallback we'll tune RandomForest.")
    estimator = RandomForestRegressor(random_state=42, n_jobs=-1)
    tune_pipeline = Pipeline([('preproc', preprocessor), ('reg', estimator)])
    param_dist = param_grids['Random Forest']
    rs = RandomizedSearchCV(tune_pipeline, param_distributions=param_dist, n_iter=20, cv=3,
        scoring='r2', random_state=42, n_jobs=-1, verbose=1)
    rs.fit(X_train, y_train)
    best_model_tuned = rs.best_estimator_
    tuned = True
    print("Fallback tuning complete. Best params:", rs.best_params_)

```



```

Best baseline model: Gradient Boosting
Tuning Gradient Boosting with RandomizedSearchCV ...
Fitting 3 folds for each of 20 candidates, totalling 60 fits
Tuning complete. Best params: {'reg__n_estimators': 100, 'reg__max_depth': 3, 'reg__learning_rate': 0.1}

```

This section selects the best-performing baseline model and applies hyperparameter tuning (using `RandomizedSearchCV`) to optimize its performance. It automatically handles tuning for multiple algorithms (like Random Forest, XGBoost, SVR, etc.) and saves the best-tuned model with optimal parameters for final evaluation.

6.8 Predictions

```
# =====
# 7) Evaluate tuned model (if tuned) and save outputs
# =====
if tuned and best_model_tuned is not None:
    print("\nEvaluating tuned model on test set ...")
    # predictions
    y_pred_tuned = best_model_tuned.predict(X_test)
    # If tuning was done for SVR using log-target wrapper, the wrapper already performs inverse transform
    y_pred_tuned = np.clip(y_pred_tuned, 0, None)

    r2_tuned = r2_score(y_test, y_pred_tuned)
    mse_tuned = mean_squared_error(y_test, y_pred_tuned)
    print(f"TUNED MODEL ({best_baseline_name}) → R²: {r2_tuned:.4f}, MSE: {mse_tuned:.4f}")

    # Save tuned model
    joblib.dump(best_model_tuned, "best_model_tuned.joblib")
    print("Saved tuned model to 'best_model_tuned.joblib'.")

    # Initialize preds_df before using it
    preds_df = pd.DataFrame({'Actual': y_test, 'Predicted_Tuned': y_pred_tuned})

    # Append tuned predictions to preds_df and save
    preds_df['pred_tuned'] = np.round(y_pred_tuned, 4)
    preds_df.to_csv("predictions_with_tuned_model.csv", index=False)
    print("Saved predictions with tuned model to 'predictions_with_tuned_model.csv'.")

    # Visuals: predicted vs actual & residuals
    plt.figure(figsize=(6,6))
    plt.scatter(y_test, y_pred_tuned, alpha=0.6)
    plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
    plt.xlabel("Actual Yield")
    plt.ylabel("Predicted Yield (tuned)")
    plt.title(f"Tuned {best_baseline_name}: Predicted vs Actual")
    plt.show()

    resid = y_test - y_pred_tuned
    plt.figure(figsize=(8,5))
    sns.histplot(resid, kde=True)
    plt.title("Residuals (Actual - Predicted) for Tuned Model")
    plt.xlabel("Residual")
    plt.show()
```

This section evaluates the tuned (best) model on the test set, saves it for future use, exports predictions to a CSV file, and visualizes actual vs predicted yields and residual errors to assess model accuracy and performance.

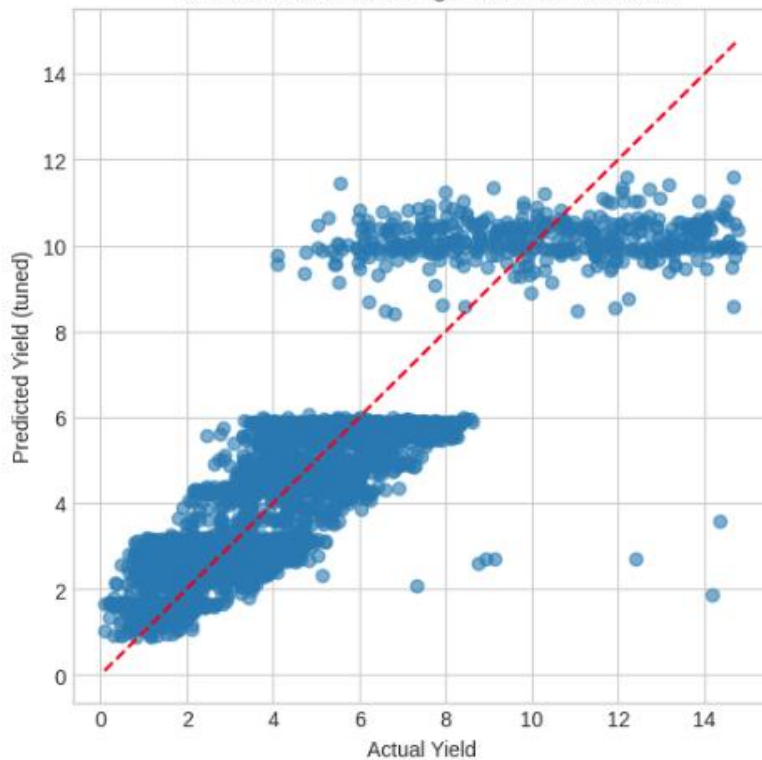
Evaluating tuned model on test set ...

TUNED MODEL (Gradient Boosting) → R^2 : 0.7733, MSE: 2.0716

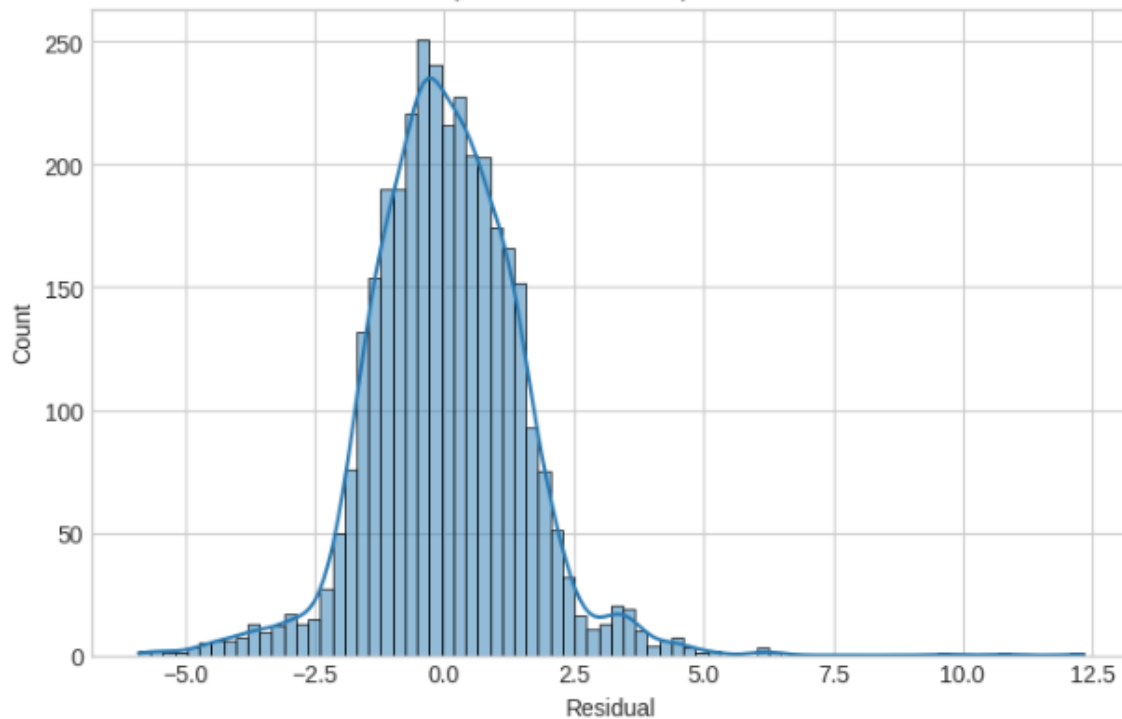
Saved tuned model to 'best_model_tuned.joblib'.

Saved predictions with tuned model to 'predictions_with_tuned_model.csv'.

Tuned Gradient Boosting: Predicted vs Actual



Residuals (Actual - Predicted) for Tuned Model



```

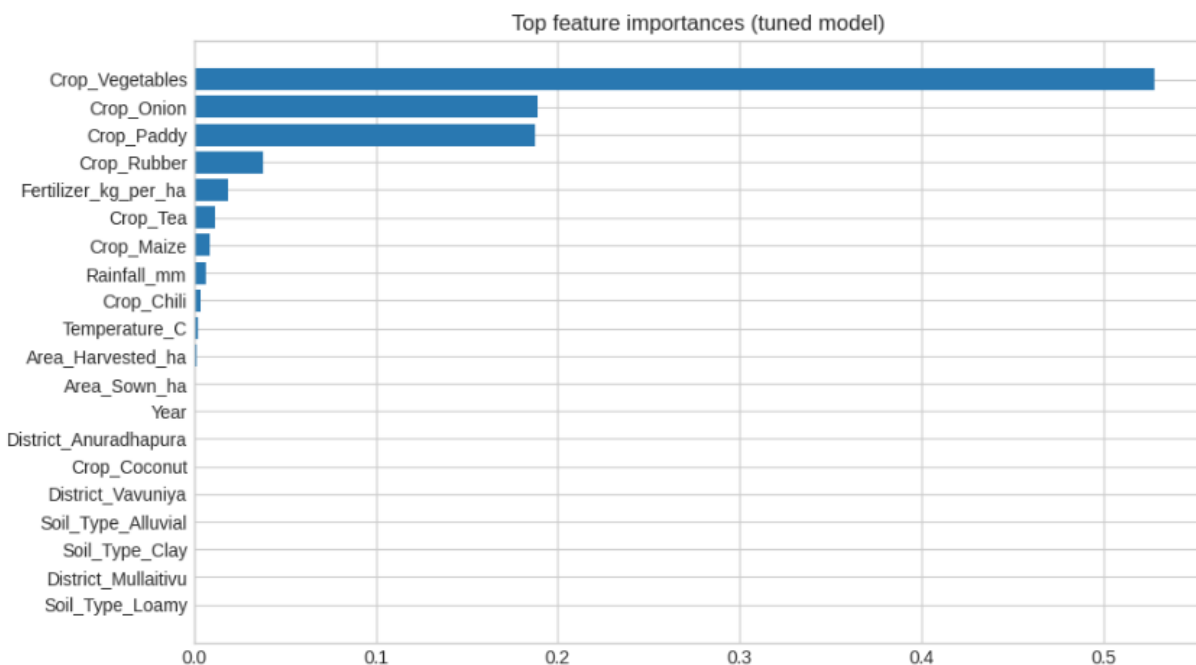
# Feature importance (if tree-based)
tree_based_names = ("Random Forest", "Extra Trees", "Gradient Boosting", "Decision Tree", "XGBoost", "LightGBM", "CatBoost")
if best_baseline_name in tree_based_names:
    # extract regressor and feature importances if available
    try:
        reg = best_model_tuned.named_steps['reg'] if isinstance(best_model_tuned, Pipeline) else best_model_tuned
        # if pipeline, get nested regressor
        if isinstance(reg, (RandomForestRegressor, ExtraTreesRegressor, GradientBoostingRegressor)) or xgb_available and isinstance(reg, XGBRegressor) or lgbm_available and isinstance(reg, LGBMRegressor) or catboost_available and isinstance(reg, CatBoostRegressor):
            importances = reg.feature_importances_
            # map to feature names (we built feature_names earlier)
            idx_sorted = np.argsort(importances)[-20::-1]
            plt.figure(figsize=(10,6))
            plt.barh(np.array(feature_names)[idx_sorted], importances[idx_sorted])
            plt.gca().invert_yaxis()
            plt.title("Top feature importances (tuned model)")
            plt.show()
    except Exception as e:
        print("Could not compute builtin feature_importances_!", e)

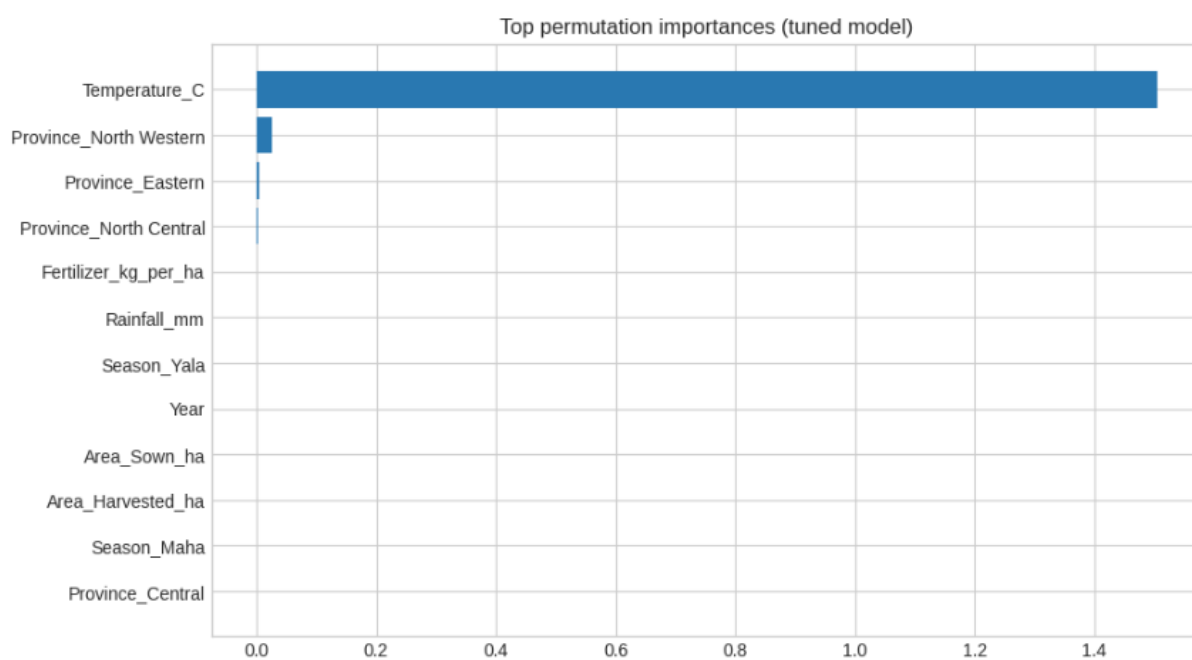
# Permutation importance (slower, but model-agnostic)
print("Computing permutation importance (this may take a while)...")
try:
    perm = permutation_importance(best_model_tuned, X_test, y_test, n_repeats=10, random_state=42, n_jobs=-1)
    sorted_idx = perm.importances_mean.argsort()[-20::-1]
    plt.figure(figsize=(10,6))
    plt.barh(np.array(feature_names)[sorted_idx], perm.importances_mean[sorted_idx])
    plt.gca().invert_yaxis()
    plt.title("Top permutation importances (tuned model)")
    plt.show()
except Exception as e:
    print("Permutation importance failed or was too slow!", e)

# Show top predicted rows for user inspection
out_df = pd.DataFrame({
    'actual': y_test.reset_index(drop=True),
    'pred_tuned': np.round(y_pred_tuned, 4),
    'abs_error': np.round(np.abs(y_test.reset_index(drop=True) - y_pred_tuned), 4)
}).sort_values(by='pred_tuned', ascending=False).reset_index(drop=True)
print("\nTop 10 highest predicted yields (tuned model):")
print(out_df.head(10))

```

This section identifies which features most influence the yield prediction using tree-based feature importance and permutation importance, then displays and visualizes the most important features and a few predicted vs actual yield values for inspection.





Top 10 highest predicted yields (tuned model):

	actual	pred_tuned	abs_error
0	14.67	11.6026	3.0674
1	12.20	11.5966	0.6034
2	5.55	11.4466	5.8966
3	13.16	11.4143	1.7457
4	12.07	11.3595	0.7105
5	9.11	11.3392	2.2292
6	12.71	11.2979	1.4121
7	12.09	11.2466	0.8434
8	7.98	11.2314	3.2514
9	10.27	11.2036	0.9336

7. User Interfaces

The screenshot displays a web application interface for predicting crop yield. It features two columns of input fields on a background image of a rice field. The left column contains dropdown menus for 'Year' (2013), 'Season' (Yala), 'Province' (North Western), 'District' (Puttalam), 'Crop' (Tea), 'Soil Type' (Alluvial), and 'Irrigation' (Rainfed). The right column contains input fields for 'Area Sown (ha)' (942.8), 'Area Harvested (ha)' (865.9), 'Rainfall (mm)' (1836.2), 'Temperature (°C)' (27.5), and 'Fertilizer (kg/ha)' (243.4). A green 'Predict Yield' button is positioned at the bottom right of the input area.

Parameter	Value
Year	2013
Season	Yala
Province	North Western
District	Puttalam
Crop	Tea
Soil Type	Alluvial
Irrigation	Rainfed
Area Sown (ha)	942.8
Area Harvested (ha)	865.9
Rainfall (mm)	1836.2
Temperature (°C)	27.5
Fertilizer (kg/ha)	243.4

Predict Yield

The screenshot shows the 'Prediction Results' section of the application. It features a green background with a yellow border. The title 'Prediction Results' is displayed in yellow text. Below the title, two green boxes with white text show the calculated values: 'Yield: 2.81 mt/ha' and 'Production: 2434.8 metric tons'.

Prediction Results

Yield: 2.81 mt/ha

Production: 2434.8 metric tons

8.Conclusion

In this project, we built a regression-based yield prediction system for Sri Lankan agriculture. Using a multi-season, multi-region dataset (20,100 records $\times$ 15 features covering crop, season, province/district, soil type, irrigation, area sown/harvested, rainfall, temperature, fertilizer, and market price), we developed models to predict yield (mt/ha)—and, where useful, production (mt).

Our workflow emphasized rigorous data preprocessing: coercing numeric-like text fields to numeric, handling missing values with sensible imputations, encoding categorical variables, checking agronomic plausibility (e.g., yield–area–production consistency), and engineering features (e.g., cumulative rainfall to date, interactions like Irrigation $\times$ Rainfall). We compared multiple regression algorithms—including linear baselines and tree-based ensembles—using cross-validation and held-out evaluation with metrics such as RMSE, MAE, and R^2 . We then applied hyperparameter tuning to improve generalization and used model interpretability tools (e.g., feature importance/SHAP) to understand the drivers of yield across crops and regions.

Key takeaways:

- Tree-based ensembles typically capture non-linear interactions between weather, soil, irrigation, and management more effectively than simple linear models, improving predictive accuracy.
- Data quality steps (type coercion, imputation, and careful encoding) substantially impacted downstream performance—reinforcing that solid preprocessing is foundational for reliable agri-ML.
- Explainability was crucial for operational adoption: insights on the relative influence of rainfall, irrigation regime, fertilizer rate, and cultivated area help planners and extension officers make targeted, mid-season decisions.

Overall, this project strengthened our end-to-end ML skills—from data preparation and model selection/tuning to evaluation and interpretation—and produced a practical, early-season yield forecasting capability. These methods can be extended into dashboards or decision-support tools to guide input allocation, import/export planning, and farmer advisories in future seasons.